In case we don't know the index of the element that we wish to delete but we do know the value of the element, we can use the `remove` method which accepts the element value as a parameter and deletes the element from the list. If the list contains duplicates, it deletes the first occurrence of the element and if the element is not found in the list it will raise a `ValueError`.

```
>>> a = [1, 2, 3, 4, 3]
>>> a.remove(3)
>>> print(a)
[1, 2, 4, 3]

>>> a.remove(9)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

Sets

`Sets` in Python can be interpreted as an exact representation of sets in Mathematics. Simply put, set is an unordered collection that contains unique objects wherein each object is immutable. A set can be initialized using the set constructor along with an iterable passed as a parameter or by providing comma separated values enclosed in curly brackets.

```
>>> set1 = set("aabbcc") # using the set constructor
>>> print(set1)
{'b', 'a', 'c'}

>>> set2 = {'a', 'b', 'c', 'a', 'b', 'c' } #implicit set declaration
>>> print(set2)
{'b', 'a', 'c'}
```

Manipulating Sets

- We can inject new elements into the set by using the `add` function along with the element parameter. If an element is already present in the set, `add` function will have no effect on the set nor will it raise an error.

```
>>> set3 = {1,2,3,4}
```